从零搭建这个 Agent：按七条主线理解整个项目

整理日期：2026-11-10。依据现有功能清单和其中标明的代码版本整理；这次只调整讲法，没有增加程序功能。上一版 完整功能清单 继续保留。

以前那份清单适合查“有哪个功能”，但不太适合理解“为什么要有这些东西”。这份换一种讲法：假设我们从一个空项目开始，手里只有一批资料，要一步步做成现在这个 Agent，会先解决什么，再解决什么。

这里说的是方便理解的构建顺序，不是在补写项目真实开发历史。先读前面的七条主线和最后那条完整问题流程，就能知道各部分怎么接；后半部分再按相同主线放全部功能明细，查细节时再看。原来的功能、限制、参数、命令和文件索引都保留。

先把整个项目串成一条路

资料进来 → 判断怎么处理 → 提取原文并记住出处 → 编码、建索引、保存知识 → 根据问题找证据 → 判断证据够不够、要不要做因果分析 → 拆任务、执行、补充和重规划 → 计算并生成带引用的报告。

这条路外面还有两圈：一圈负责让任务出错后能恢复、能取消、能查记录；另一圈负责准备训练数据、改进本地模型和检查效果。训练不是每次回答问题都要跑的一步。

可以先记住七条主线：

主线	从哪里开始	最后交出什么
一：资料进来	PDF、文本、结构化法条	可以处理的原文、出处，或明确的待处理状态
二：知识留下来	已提取的原文	能检索、能按目录读、重启后还在的知识
三：证据变成分析依据	问题和检索结果	提取事实、充分性判断、需要时的因果图和权重
四：分析变成任务流程	一个具体问题	有顺序、有并行、有重试的执行过程和最终报告
五：流程变成可靠服务	一次次实际运行	保存状态、恢复、回滚、API 和消息入口
六：准备数据和改进模型	已有任务定义、来源和训练脚本	合成样本、待审候选、可选模型适配器
七：检查和维护	每条主线的输入、结果和记录	测试、评测、日志、配置及明确的能力范围

主线一：先弄清资料是什么，再决定怎么处理

第一步：不要让所有文件一进来就走最复杂的流程

我们最先面对的不是“要建什么图”，而是“手里来了什么资料”。结构化法条、案件材料、参考文档和模型生成的总结，用途不同，处理成本也不一样。

因此先做轻量分流。已有法名、条号和正文的 JSONL，能认作结构化法条；PDF 可以由调用者声明是规范资料、案件资料还是参考资料。程序还会看有没有现成结构、页数、文字提取是否有问题。这里不是让另一个大模型替我们评重要性，而是用现有字段和固定规则决定加工深度。

第二步：能直接用的简单处理，确实需要的再深加工

短而且文字清楚的资料，保留原页和基础索引就够了。没有结构的长案件 PDF，才需要进一步建章节目录；摘要也不是强制每份生成。参考资料不自动套复杂层级。派生内容先归档，用途不明的先保留，避免一股脑塞进证据库。

如果是普通文本，则先按段落或句子切块，合并过短片段，再输出节点和向量。这个文本目录入口支持 txt、md、text，不是递归扫描所有 PDF 的入口。

第三步：先确认能读全，再谈入库

PDF 先提取原生文字，检查空文本页、异常字符，也可以主动打开 pdf-inspector 检测。需要 OCR 或版面太复杂、目前读不可靠，就返回待提取状态和问题页，不把缺页内容当成完整证据。

当前没有自动 OCR 和复杂表格解析。这不是“模型再猜一下就能补上”的环节，资料得先处理到可读。原文件始终保留，路径、哈希、文档 ID 和物理页码也要跟着走。

这一条线的结果：下一条线拿到的是有出处的原文；没处理好的资料，也有明确状态。它还不是事实结论，更不是因果关系。

主要代码：material_routing、material_rules、pdf_intake、chunk、prepare_corpus。完整细节在后面的“资料进来”明细。

主线二：把原文组织好，存下来，让后面的任务能找到

第一步：先解决“怎么找到相近的资料”

原文准备好后，用 BGE-M3 编成向量，交给 FAISS 做近邻检索。igraph 另外记录原文、摘要、相似关系和附带信息，这样不仅能找最近的片段，还能沿相关节点补查。

这就是语义知识图的用途：帮我们找相关内容。相似只是相似，不会因为两段话连了边，就自动变成“前者导致后者”。

第二步：长文档再加目录，摘要用来找路

一本长文档仅靠碎片相似度，可能不容易定位上下文。于是参考 PageIndex 的思路，先读书签，再用标题和编号补出章节结构；需要时从小节到章节生成摘要。

目录和摘要负责告诉 Agent 往哪里找，真正的证据还是原页。Agent 可以列文档、看目录、读指定页，也可以退回向量检索。短文档在预算内能读完就直接读，不必绕一圈工具调用。

这里没有整套移植 PageIndex。现有成本组件只估算和建议，不会自动替我们展开或合并目录。

第三步：知识更新后，相关摘要也要跟着变

新资料可以加入语义图，挂到合适摘要下；旧资料可以软删除。内容变了，相关摘要会标为待更新，再按先子后父的顺序重算。

另外还有独立的 GMM 增量记忆组件，用摘要相似度判断新知识挂到哪些簇。MemoryGraphBridge 负责把它和实际图引擎接起来，尽量让同一条资料只编码一次。这条增量接口不是每次问答都必须调用的步骤。

第四步：不能重启一次就把知识丢掉

原文、摘要、来源、向量和图关系会保存进 SQLite 长期记忆。后续任务或进程重启可以恢复，不用再让模型生成一遍。不同项目按项目 ID 分开，换编码器还要换向量空间版本并重新入库。

这里存的是知识。任务做到哪一步，由后面的任务运行库负责；别把两种“记忆”混为一谈。

这一条线的结果：得到可以长期保存、向量检索、按目录阅读的资料库。它交给分析模块的是原文和出处，不是模型摘要冒充的证据。

主要代码：graph、summary_tree、IncrementalMemoryManager、memory_graph_bridge、long_term_memory、document_index、document_store、document_navigation、page_index_memory。

主线三：找到资料以后，判断它能不能支持回答

第一步：围绕问题读原文，不是把整个库都塞给模型

分析从一个具体问题开始。系统可以先向量召回、沿语义图扩展，也可以让 Agent 按文档目录找原页。轮数、页数和读取 tokens 都有限制，读到哪、怎么读到的会记下来。

之后，Grader 从原文提取事实，判断资料足够、还缺一部分，还是完全不相关。它给评分的理由会单独保存，不直接当作法律推理。

第二步：先区分“缺资料”和“缺用户事实”

如果缺法律或其他来源证据，就继续查或换检索方式。如果缺的是用户自己的情况，就停下来问用户。用户补充可以帮助分析，但不会被写成“原文里查到的事实”。

普通事实任务到这一步就可以把结果交回执行流程。只有任务要求推理，而且证据足够时，才单独调用 Reasoner；并不是每个小任务都要把所有模型轮流跑一遍。

第三步：只有问题需要因果分析时，才加 CHAIN

比如问题需要判断某件事是否导致另一件事，我们才把读到的原文再切成 512-token、相邻重叠 64-token 的块，让模型提取实体、事实命题和三类关系：导致、创造条件、阻止。

程序先查端点和引句，再用一次独立模型请求判断：原文明确支持因果，还是仅仅相关、假设、否定或根本没支持。通过的关系由程序组装成因果—时间超图。这个图不是 GNN 训练出来的，也不是从语义相似边直接变出来的。

第四步：时间权重和支持分各管一件事

给了明确截止日和目标，才进一步筛选当时可用的证据，从目标反向 BFS 算因果距离，再结合时间衰减、重复支持和已有相似度算 CTVF，最后累乘得到因果链权重。

另一个独立问题是模型打的支持分准不准。校准组件可以用人工标注拟合曲线，保留原始分和校准分；没校准就明确标出来。文本支持分、时间有效性、现实因果效应和法律效力，不是一个概念。

这一条线的结果：给执行流程提供有来源的事实、充分性判断、缺失信息，以及按需生成的因果分析。不需要因果的普通问答可以跳过整个 CHAIN 环节。

主要代码：retrieval、reasoning、evidence、operators，以及 causal_evidence 下的分块、提取、验证、建图和校准模块。

主线四：把一次分析，组织成能完成问题的 Agent 流程

第一步：先决定问题需不需要拆开

闲聊直接回复，简单问题直接查资料，复杂问题交给 Planner。Planner 先写清要做哪些事、谁依赖谁，再把它们具体化成检索任务，也就是两层计划。

每个任务带任务 ID、依赖、检索引擎和证据策略。证据策略允许组合：要不要目录导航、要不要因果验证、是否继续推理，不用为每种组合再造一个角色。

第二步：能独立做的同时做，有前置条件的等结果

Executor 找出依赖已完成的任务，选一批并行执行。每个分支有自己的重试和读取记录，但共享本次任务的只读资料版本。做完后按计划顺序合并，已完成的结果先存下来。

比如一个问题需要查两类互不依赖的资料，就可以同时查；需要前一个任务事实的计算，则要等前一个完成。当前一批做完才进入下一批，还不是连续抢占式调度。

第三步：失败后改失败的部分，不把所有事情重做

资料不够可以重试；检索不相关或多次失败，再让 Replanner 换词、换任务或改成全局向量“虫洞”。已经完成的分支尽量复用。

合并后还会用七类现有硬规则检查有没有遗漏，比如是否漏了某类请求的核查。规则补出来的任务有固定 ID，避免恢复时反复追加。缺用户信息则挂起，补充后继续。

第四步：计算交给程序，报告回到原文

需要金额时，默认让模型给结构化计算计划，Decimal 执行加减乘除。只有主动允许代码执行，才生成 Python，放 Docker 或可选 SPACE 沙箱运行，并限制重试和资源。

最后 Generator 汇总观察、事实和计算结果。如果用了版本化文档，回答还得给出读过的证据编号和连续原文引句，程序检查出处、页码和引句。引用对得上，仍不自动证明整段结论在语义上成立。

这一条线的结果：一个有任务拆分、并行、补充、失败处理、计算和引用的完整问答流程。

主要代码：double_layer_plan、workflow、nodes、scheduling、operators、replanner_rules、calculator、sandbox、citations。

主线五：把能跑的流程，做成能长期使用的工程系统

第一步：让所有默认入口进入同一套服务

上面解决的是“怎么做事”。再往工程上走，CLI、HTTP 和 Kafka 默认都交给同一个 AgentService，才能统一状态、预算和取消规则。HTTP 提供共享令牌保护，Kafka 只负责通知，不把消息内容当任务状态的唯一依据。

旧五阶段 Worker 和 Redis 状态链还保留兼容，但它们不是新的默认运行方式。旧 Kubernetes 示例也不能直接当作新系统的多机部署方案。

第二步：把任务做到哪里、工具做过什么存下来

运行 SQLite 管状态、预算、事件和工具记录；LangGraph 检查点库管流程位置。执行者要续租，写回时检查租约和 epoch，避免旧进程在任务取消或回退后继续改结果。

任务还会固定一份知识快照和模型 / 提示配置。重启后既要从同一个步骤继续，也要确认用的是同一套证据和执行设置。

第三步：分别处理重试、恢复和回滚

重试是在允许的条件下再做；恢复是进程退出后接着做；回滚是选择旧检查点退回去。工具结果如果已经存好，可以在重放时直接复用。Kafka 待发通知也先存 outbox，确认发出后才标记完成。

但这些能力撤销不了外部世界：模型调用可能已经计费，沙箱创建请求可能已经生效。结果不明时要保留记录核对，不能靠回退状态就假装没发生。

第四步：限制资源，再加备份

同机同时跑几个问题、模型请求、编码和沙箱，都有独立额度。文件锁协调进程，GPU 拥有者锁避免多个进程各加载一份权重。这不是显存自动卸载，也不是多 GPU 放置器。

最后再备份运行库和检查点库，检查哈希和完整性。知识库、原始 PDF 和模型文件另备份。日志、事件和错误类型用于定位问题，恢复则只写新目录，不直接覆盖现有库。

这一条线的结果：流程不再只靠一个脚本内存活着，而是能提交、查询、等待、取消、续跑、回退和备份的单机服务。

主要代码：agent_runtime 全部模块、asynchronization 兼容链、observability、配置和部署文件。

主线六：再准备训练数据，逐步改进本地模型

第一步：先接好模型，再考虑训练它

默认 API、本地 vLLM、不同角色模型和 LoRA 都从已有调用接口接入。前缀缓存只是复用计算，旧语义答案缓存是复用答案，长期记忆是保存知识，检查点是保存任务进度，四者不要混用。

工具规划出错，也不一定第一步就训练模型。PA-Tool 先让目标模型给工具和参数取熟悉的名字，再把结果翻译回项目原接口。它改的是工具 Schema，不改权重，而且目前还没用真实目标模型测出收益。

行为优化组件则是另一回事：离线准备和调用 Heretic，尝试减少指定迎合词、套话，另存候选 LoRA。它默认不运行、不自动挂载，也不处理最终 Generator API；关键词变少不等于推理更可靠。

第二步：数据要分成“程序能验”和“还得审”

新的合成组件可以先造小环境，生成查询、汇总、依赖、并行交接、失败恢复和约束任务，再由程序计算正确目标、回放参考步骤。任务比例、复杂度、重复和冲突都能控制，训练 / 验证 / 测试先按来源组分开。

法律、因果和开放推理题不能因为模型生成了一个答案，就叫可靠标签。它们进入 pending 待审区，和模拟规则内 verified 的样本分开。轨迹与排序对也能导出，但程序脚本轨迹不是模型真的执行过的日志。

旧 Evol-Instruct 仍可根据法条和判决生成题目、计划与重规划场景，不过处理错误和验证标签的方式不同，不能直接当成新组件的已验证数据。

第三步：最后才是各角色训练

现有五条训练代码分别负责 Planner 的 SFT、Replanner 的 GRPO、Extractor / Grader 的 DPO、Reasoner 的 IRAC SFT，以及 Retriever 的对比学习。加载器支持 4bit QLoRA、梯度检查点和单独保存适配器。

但合成组件还没有自动把每种输出转成五套训练输入，训练脚本也没有全部经过真实 GPU 训练验证。尤其当前 GRPO 的模拟检索和代理奖励，不等于真实任务完成奖励。要把训练闭环做可靠，还得补接数据格式和独立验收；不能说生成完就已经能自动训练出更好的 Agent。

这一条线的结果：得到独立的数据生成、接口适配、行为优化和训练工具。它们是改进系统的工具箱，不是每次在线问答都运行的流程。

主要代码：llm、vllm_engine、schema_alignment、tool_schema、behavior、model/data、model/training、unsloth_loader。

主线七：每走一步，都要知道它有没有真的变好

第一步：先检查程序能不能按约定工作

导入检查、语法编译、pytest、离线 demo 和跨进程恢复实验，主要回答程序是否遵守接口、是否能恢复、失败是否按约定处理。这里大量使用确定性替身，让测试不依赖 API 额度和外部服务。

这些检查应贯穿前面每条主线，不是把项目全部搭完再测一次。它们也不能替代真实模型和部署检查。

第二步：再检查检索和分析结果

检索有 HR@K、MRR、噪声占比和 Graph-NIAH；执行过程可以交给裁判模型评分。离线 SCM 另外用明确结构方程构造观察、干预、反事实题，用程序核对引用、日期、金额和关键结果。

SCM 是测试环境，不能直接搬成线上事实。词法匹配和模型评分也各有局限，指标存在不代表已经测出真实效果。

第三步：优化速度前，先留住原来的实现

批量编码和快照索引缓存已经有可选代码，默认关闭。它们分别尝试减少小批调用和重复构建索引，不是 CUDA / Triton 算子。需要先对照结果、测真实耗时和峰值内存，再决定开不开。

配置、日志、指标、工具记录、版本和测试记录，都是为了能说清“这次换了什么、有没有变好、哪里失败”。当前没有完整训练效果、真实 GPU 性能或全部外部服务联调结论，文档也会保留这些区别。

这一条线的结果：项目能检查、能追查，也能区分写好了代码和实际验证了效果。以后加新功能时，仍能沿这七条线找到它应该放在哪里。

再用一个问题把七条线连起来

假设用户问：“这些材料够不够支持我的请求，还需要补什么？如果涉及金额，怎么算？”这里只演示系统流程，不预设法律结论。

1. 先准备资料。 原生文字 PDF、结构化法条和参考材料走合适的轻重处理；提不全的先标待处理，出处一起保存。

2. 让知识可查。 编码、建索引、保存原文和目录。跨任务使用的是这套长期知识，不是重新凭空生成资料。

3. 接到问题。 AgentService 建运行，记录预算；复杂问题交给 Planner 拆任务，并固定本次资料版本。

4. 按任务找证据。 有的分支查向量，有的读目录原页；互不依赖的可以同时做。

5. 判断够不够。 Grader 提取事实，缺用户信息就等补充，缺资料就检索或重规划。只有因果问题才按需加 CHAIN；只有要求推理且证据足够才单独调用 Reasoner。

6. 计算并回答。 默认按结构化计划用 Decimal 算；允许代码时才进沙箱。Generator 汇总结果，并检查引用确实来自本次读过的原文。

7. 中途退出也能处理。 已存工具结果和检查点帮助恢复；取消或回退提高执行轮次，拦住旧写入。外部操作是否已生效仍要另核对。

8. 以后再改进。 用评测和日志找到问题，再决定补资料、改工具名字、造可验证样本，还是训练某个角色。这一步不自动发生，也不会把本次回答直接当成可靠训练标签。

几种图和几种存储，放回各自的位置

名字	回答什么问题	不要把它当成什么
语义知识图	哪些资料相近，还能顺着找什么	现实因果关系
文档目录树	这份资料哪个章节、哪几页值得读	已验证的结论或原文证据
文本因果—时间超图	原文声称什么因果，支持程度和时间权重怎样	真实因果效应统计估计
离线 SCM	在人工写明的方程规则里，干预后应发生什么	自动学出的现实世界模型
任务 DAG	先做什么，哪些事可以并行	又一份证据数据库
长期知识库	原文、摘要、来源和向量怎么跨任务保存	当前任务运行到哪一步
运行库和检查点库	本次任务状态、工具记录和流程位置	可撤销外部世界操作的事务系统

前面几种图不是要求原文分别复制进四份显存。它们表达不同关系，目录和语义关系会复用原文及文档版本；CPU 向量、运行快照仍有重复占用，不能说完全去重。

完整功能明细：按七条主线查

下面把上一版的完整明细重新分组，功能条目和表格保留，不要求一次看完。前面的主线用于理解，后面的明细用于查功能、参数和代码。第七组也保留默认值、命令和未实现事项，最后附全部源码和测试索引。

明细一：资料进来

怎么处理普通文本

代码在这里：chunk、prepare_corpus。

- 旧 PDF 分块工具可以提取文件里的文字，再在命令行预览分块。它读的是全文，不会像新导入流程那样管理文档版本。
- 可以按自然段，也可以按中英文句末标点切块；太短的片段会合到一起。
- 设置最大字符长度后，还能按句子继续拆。但它不保证严格的 token 上限，也不会强行切开所有超长句。
- 文本目录支持 .txt、.md、.text。按文件名排序后处理，保证同样的输入能得到同样的节点 ID。
- 这个目录扫描不会往子文件夹里继续找，也不是批量 PDF 扫描入口。
- 每个节点会存整数 ID、正文、Raw 类型、源文件名、块序号、字符长度、原文指纹和 record/source ID。
- 可以用图引擎默认编码器、自己传入的单条编码器，或者离线用的 StubEncoder 生成向量。
- 输出 nodes.jsonl 和 vectors.npy，节点和向量按同一行序一一对应。
- 已有向量可以直接拿来建图，不用再算一遍。
- 输入目录可以通过 CORPUS_DIR 或命令行参数指定。目录不存在、里面没有支持的文件，都会报错。

资料怎么分流，PDF 怎么导入

代码在这里：material_routing、规则、pdf_intake、说明。

- 资料分流用固定版本的 rule-engine 执行规则文件，不用再加载一个语义分类模型。
- 先看资料用来做什么、有没有现成结构、有多少页、文字好不好提取，再决定处理多复杂，不只看一个“重要性分数”。
- 规范资料 normative：已有法条结构，或者短而且能直接读的，简单处理；文字难提取，或者是没有结构的长 PDF，再做深加工。
- 案件资料 case：短而且好读的，简单处理；难提取或没有结构的长 PDF，再做深加工。
- 参考资料 reference：保存原文页、做好基础索引就行，不自动加复杂目录。
- 模型总结等派生资料 derived 会返回 archive，先归档，不自动放进原始证据库。
- 不知道用途的资料返回 deferred，原件保留，等确定类别后再处理。
- JSONL 里已有非空法名、条号和正文时，可以识别为结构化法条；如果明确标成派生资料，就先按派生资料处理。
- 不会只看 PDF 文件名或标题就猜它重不重要。调用时可以用 material-kind 指明资料用途。
- 缺来源或时间会加上 review_flags，提醒后面检查。旧版法条也保留，不改原文。

- JSONL 一条条处理，报告只记源路径、文件行号、记录哈希、类别、处理方式、规则 ID 和版本，不再复制一整份正文。

- 报告先写临时文件，全部成功后再一起换成正式文件，也不允许覆盖原始资料。

- PDF 也是先分用途。如果用途未知或属于派生资料，就不会启动 PDF 解析器和图引擎。

- 先检查 PDF 能不能直接提文字，找出空文本页和异常替换字符；也可以主动启用 pdf-inspector 做检测，不做 OCR。

- 检查器的页码从第 1 个物理页开始算。它发现的问题会和原生检查合在一起，不互相覆盖。

- 简单处理只保存原文页和空目录，不做层次摘要，也不调用摘要模型。不过，基础向量编码和图仍然会用内存。

- 深加工会复用现有目录构建器；只有你明确要求时，才让模型生成摘要。

- 如果需要 OCR，或者复杂版面还读不可靠，就返回 pending_extraction 和有问题的页码 / 检查结果，先不把缺页内容当成完整证据入库。

- 不会改动或移动原 PDF。导入后会记住源路径、文件哈希、文档 ID，以及每条内容对应的物理页码。

- 目前还不能自动做 OCR、PaddleOCR、复杂表格提取，也没有自动复核队列和重试服务。

- 旧 import-pageindex 入口照常保留，但不会自动走这套新分流规则。

明细二：知识留下来

知识图怎么组织，摘要怎么更新

代码在这里：graph、IncrementalMemoryManager、桥接器、摘要依赖。

- BGE-M3 同时生成 dense 稠密向量和 sparse 词权重。FAISS 负责查向量，igraph 保存节点、语义边和附带信息。

- 节点用整数 ID，igraph 用字符串名称；两者通过同一套规则转换，避免查到别的节点。

- 比较相似度时会结合向量余弦和稀疏词权重；没有可用稀疏信号时，就只看余弦。

- 可以一次批量建图，也可以直接传已有向量；近邻分批查，是否连边由相似度阈值决定。

- 可以逐条加 Raw 原文、Summary 摘要等节点，再选合适的节点连边。特别相似的会标 similar_to，但不会因此删掉原文。

- 新原文可以挂到语义接近的摘要下面，并提醒上层摘要：内容变了，需要更新。

- 节点连边太多，或者簇变大时，会先记下需要处理。不会每加一条资料，就马上让模型重新聚类。

- 软删除会保留 tombstone 标记，查询时跳过它，同时通知相关摘要更新、刷新索引。

- 摘要会记住自己来自哪些子节点。内容变了以后，可以只重算相关摘要。

- 生成层次摘要时先做下面的小节，再做上面的章节；父章节会同时看子摘要和还没被子节点覆盖的正文。

- 同一页属于多个章节时，这些章节都可以标为待更新。原文删掉后，也能清除失去来源的摘要。

- 批量改摘要时，只要中间失败，就不会把半新半旧的结果发布出去。

- 文档章节按原目录挂载，不会被普通语义聚类随意换位置。

- 独立的增量记忆组件会用 GMM，把与新资料相似的摘要分组，挑出可以接收它的簇，同时要求相似度不能低于下限。

- 样本太少、分数没差别或 GMM 拟合出问题时，旧组件会改用中位数阈值。这部分确实有兜底处理。
- 一条新知识可以挂到多个合适的簇；一个都不合适时，就建新簇，把第一条叶子也挂进去。
- 有“夜间重写摘要”和“重建摘要索引”的调用接口，但目前没有服务每天晚上自动执行。
- 独立增量组件用的是内存里的 MockGraphDB。不传摘要函数时，只会用演示用的文字拼接，不会自动生成高质量摘要。
- MemoryGraphBridge 会把同一次编码的结果同时交给增量记忆和图引擎，避免同一条资料算出两份不同向量。
- 桥接器可以从已存图重建状态、加入新知识、同步待更新摘要、重试未同步内容、做软删除和查看统计。
- 可以加载图编码器 LoRA 或 SentenceTransformer 格式的产物，但要确认它和实际编码器、向量空间版本一致。

知识怎么跨任务、跨重启保留

代码在这里：long_term_memory、graph、说明。

- 知识图会存进 SQLite，用 JSON + zlib 保存快照。换任务或重启进程后，还能恢复。
- 快照里有原文、摘要、附带信息、向量、语义边、软删除、待更新摘要和待聚类标记。
- 不同项目用 project_id 分开保存，每次提交还会记下知识版本 revision。
- 写入前会比对版本。如果另一个写入者已经更新了知识库，旧写入会报 MemoryConflict，不能盖掉新资料。
- 多次写入可以放在 memory_transaction 里一起提交；中途出错就恢复修改前的图，并把错误抛出来。
- 恢复时直接用存好的向量重建索引，不重新编码，也不调用模型重写摘要。
- 新检索可以刷新知识库已保存的内容；正在跑的持久任务仍用自己的固定快照，不会半途换资料。
- 本地没保存成功的修改会保留。下一次刷新不能悄悄丢掉它们，除非调用者明确选择放弃。
- 会核对模型、维度、建图参数和 embedding_space_id。对不上就报错，不自动把不兼容的资料迁过去。
- 换模型或 LoRA 后，需要换向量空间标识，以及对应的库或项目分区，再重新入库。
- 项目知识库主要保留最新快照，不保存完整的历次知识版本。单次任务自己固定的快照是另外一回事。
- 已经同步进图的内容会保存；增量记忆里还没同步的草稿簇和队列，不在这份快照里。
- Agent 的回答和模型猜测不会自动变成原文事实。原始 PDF 文件本身也不会复制进知识库。
- 命令行可以检查知识库、列文档、读目录、读原页和导入文档。
- 这套存储按同一台机器的持久磁盘设计，目前没有多机共享存储、跨机事务或网络共享盘的部署保证。

怎么按目录找资料，怎么保留引用

代码在这里：document_index、page_index_memory、document_store、document_navigation、citations。

11.1 目录怎么建、怎么导入

- 可以导入 PageIndex 风格的根节点列表或 structure JSON 树，也能直接从带文字的 PDF 建目录。
- 本地建树用 PyMuPDF：先用有效书签搭主框架，再用章节编号和字体标题补细节，并检查标题、页码、顺序和层级。

- 目录会记下来源：书签 bookmarks、检测 detected、混合 hybrid、平面 flat，或者外部导入；发现的问题另记 diagnostics。
- 找不到可靠标题，就保留平面结构，不硬编复杂目录。
- 每个章节节点会存章节 ID、标题、起止页、子节点、摘要和 key_items 小节标题。
- 会检查章节 ID 重复、起止页写反、页码超范围，以及子章节跑到父章节范围外的情况。
- 原文页存成 Raw 证据，目录存成 Summary 导航节点，再用 source_node_ids 把它们连起来。
- 同一文档 ID 下，内容、目录和附带信息完全一样，重复导入也不会重复建节点。改了资料就要用新版本 ID。
- 缺摘要时，可以自己传摘要函数，也可以主动让模型生成。不启用生成时，就用已有摘要或标题来找资料。
- 导入中途编码失败或摘要生成失败，会撤回这次导入的修改。
- 成本诊断会看沿目录查找和直接读页各要多少工作，也考虑子节点没覆盖的页，再给出最坏分支成本和合并建议。
- 目前只给建议，不会自动改树，也没有自动 Merge / Expand。这个页数估算还不是实测的延迟或 token 成本。

11.2 怎么查看文档和原页

- DocumentReader.from_file 可以只读打开知识库，不会创建缺失的库，也不用加载 BGE、FAISS 或 LLM。
- from_engine 则从当前图引擎拿一份和本次任务一致的读取内容。
- 可以列出文档、查看目录，或者指定文档版本和物理页码来读原文。
- 文档版本由内容指纹标识。找错文档、版本、页码，或者读到已删除页，都会报错。
- 目录可以按层和偏移分页读；模型不能在参数里随便指定宿主机文件路径。

11.3 Agent 怎么一步步找资料

- 模型可以选五个动作：list_documents 列文档、read_structure 看目录、read_pages 读页、vector_search 查向量、finish 结束。
- 自动模式下，能在预算内完整读完的短文档就直接读；长文档或混合资料让模型按需找；没有版本化文档就继续用向量检索。
- 也可以手动选原来的向量检索作对照，或者直接启用 Agent 导航。
- 模型最多问几轮、最多读几条原文 / 几页、原文和目录各读多少 tokens，都能分别限制。
- 同一页重复读不会重复计数，空白页也算一页。一次请求超限就整批拒绝，不把一页截掉一半还说读完了。
- 会记下选过的动作、模型调用次数、读取量，以及无结果 empty、轮数用完 turn_budget_exhausted 等状态。
- 轮数用完后，只返回前面真正读到的原文。动作不合法、版本错误或模型调用失败，会直接报出来。
- 不管沿目录找还是用向量找，最终都要把读到的原文交给 Grader 或因果组件。

11.4 原文引用

- 给本次实际读过的证据编上 E1、E2 等编号，同时保留文档、版本、页码、记录和块 ID。
- 模型回答时，要同时给出引用 ID 和从原文里连续摘出的句子。
- 程序会检查：编号是不是本次读过的、正文有没有用到、有没有重复或漏配，以及引句是否真的存在于原文。

- 最后把编号、原文件名、页码和引句整理到答案里，同时返回可供程序使用的 citations。
- 普通证据和因果证据都走同一套引用表；摘要不能冒充原文。
- 这些检查能确认“出处对得上、原文有这句话”，但不能单靠它证明这句话足够支持整个结论。
- 这些功能参考了 PageIndex 的思路，没有安装它的 SDK、复制它的实现，也不需要它的云服务。

明细三：证据变成分析依据

怎么找证据、判断够不够用、生成报告

代码在这里：retrieval、reasoning、operators、evidence。

- 先用 BGE-M3 把问题变成稠密向量，再用 FAISS HNSW 找可能相关的资料。
- 普通图检索还会沿候选附近的语义边继续找；全局稠密检索只做向量查找，不往附近扩展。
- 软删除的节点不会返回；索引为空时直接结束，不先加载编码模型。
- 找到摘要后，会沿来源关系回到原文，再整理结果。找不到关联原文的摘要，不能直接当证据。
- 长期记忆和因果分析的查询会用知识图自己的编码器，避免拿路由模型的向量去查另一套向量空间。
- evidence.navigation 决定自动选择、只做向量检索，还是让 Agent 按文档目录查找。
- evidence.causal 决定自动判断、关闭因果、只验证文本因果，还是加时间权重；analysis 决定只提事实还是继续推理。
- 自动模式会参考截止日、目标实体和问题里的因果词，判断要不要提取因果关系。普通法条问答可以跳过。
- 有截止日时，会先过滤原文，再把能用的资料交给事实判断和因果提取。
- 会记下证据是哪条记录、来自哪里、文档哪个版本、哪一页，以及导航时怎么找到它。
- Grader 负责提取事实、判断够不够回答问题。它解释评分的理由会另存，不直接当成法律推理。
- 只有设置 analysis=reasoning，并且证据足够时，才单独调用 Reasoner。
- 最后的报告会汇总已经做完的观察、普通 / 因果证据、用户补充，以及程序算出的结果。
- 资料带文档版本时，会生成并检查结构化引用。旧的无版本法条问答仍可以返回普通文本。
- 资料不够、找不到目标、检索为空、模型出错、格式不对，会分别记清楚，不会都写成回答成功。
- 你补充的是案件事实，不能冒充 CHAIN 从原文里读出的证据。

CHAIN 怎么提取因果关系、给证据加权

代码在这里：causal_evidence、校准 CLI、方法说明。

12.1 怎么分块、提取事实和因果关系

- 每条 EvidenceRecord 会记文本、记录 ID、来源 ID、时间、可用性核实信息、相似度和原文位置。
- 默认用 cl100k_base 按 512 tokens 切块，相邻块重叠 64 tokens，并记下它在全文里的 token 和字符位置。
- 中文和 emoji 等字符不会从字节中间切开，因此实际重叠有时会略少于 64 tokens。
- 每块让 LLM 提取实体、事实和因果关系；关系只收 causes 导致、enables 创造条件、prevents 阻止这三种。
- 因果边两头必须用已经提取的实体，还要保留原始支持分、证据引句和对应的事实命题。

- 程序先检查实体、引句和命题关联，再把通过检查的候选交给一次独立 LLM 请求，判断原文到底支不支持。

- 验证时会区分明确因果、只是相关、被否定、假设以及没有支持的关系。被拒绝的候选和理由也会留下。

- 实体名称只做 NFKC、大小写和空白整理，不自动猜哪些词是同义词。

- 最后由程序把实体、事实超边和有向因果边组装起来，不用 GNN 训练。

- 图里会保留来源文本块、原文位置、文档版本、页码和拒绝记录，方便回头检查。

12.2 怎么筛时间、算 CTVF 和因果链权重

- 只解析明确的日期。给到月份时按月末算，日期有歧义就当作缺失，不猜一个。

- 如果已知的事件时间或资料可用时间在截止日当天及之后，这份证据就不能进入后续可用图。

- 缺事件时间时，必须有外部核实的 availability_verified=true，且 available_at 早于截止日。模型不能自己说“这份资料当时已经可用”。

- 先筛掉不能用的证据，再从明确的目标反着沿因果边做 BFS，算每个实体离目标最近有几步。

- 再结合资料距截止日多久、离目标几步，计算时间衰减。

- 同一事实有多少来源支持、已有语义相似度多高，会影响额外支持分；没有相似度就不凭空补分。

- 时间分和支持分相加后，最多算 1。缺时间的资料用配置里明确给出的有效性分。

- 同一来源、同一事实，只是因为分块重叠出现多次，不会重复给端点加支持。

- 用边两端的平均有效性和距离算出调整系数，再乘上原始或校准后的支持分。

- 找出最后通向目标、途中不重复节点的链，把每条边的权重乘起来；遇到 prevents，还会改变链的正负作用方向。

- 链不能无限找：深度、扩展次数、返回条数和最低分都有上限。没找完也会标出搜索被截断。

- 目标没有对应实体、或者没有能用的证据，会明确返回状态。没给截止日时可以分析一般证据，但不会算预测用的时间权重。

- 这些分数表示原文支持程度和时间有效性，不是真实因果效应、事件发生概率，也不是法律效力分。

12.3 怎么校准模型打的支持分

- 模型给出的原始分数存在 raw_support。拿人工标注拟合 isotonic 校准曲线后，才会有 calibrated_support。

- 还没准备校准器时，会标 uncalibrated，校准值留空。校准后也允许把不可靠的边降到 0 分。

- 校准样本要有来源 ID、模型原始分数，以及人工判断“原文支持 / 不支持”的标签。

- 训练和留出检查按来源分组。同一份资料的重叠块、改写和转载要放在一组，不能一部分用来训练、一部分用来测试。

- 拟合前会检查来源够不够、有没有正反两类标签、分数有没有变化。留出的标签只用来检查效果，不拿回来重新拟合。

- 结果会保存校准曲线、标签哈希、来源怎么分组、用的哪个提取器，以及标注从哪天起可用。

- 会对比留出数据校准前后的 Brier 和分箱 ECE，用来检查这些支持分是否更可靠。

- 换了提取或验证模型配置，就不能直接套用旧校准文件。做历史预测时，标注也必须在预测截止日前已经可用。

- 同步流程和新的 Kafka 流程共用这套因果工具。但多问一次验证模型，仍不等于从数学上证明现实因果。

- 这里只用了 CHAIN 的建图和数据权重，没有加入 Noisy-OR 或它的整套预测融合。

明细四：分析变成任务流程

这个项目能做什么，代码怎么分工

代码在这里：README、模块职责、运行服务、流程。

- 这是一个给个人或熟悉的小团队使用的法律 RAG Agent。你提出法律问题，它会查资料、拆任务、找证据、判断信息够不够，必要时追问，最后推理并生成报告。

- 代码按用途分开：legal_agent 负责做事的流程，agent_runtime 负责保存和继续任务，agent_tools 放计算和沙箱工具，dataset 管资料，causal_evidence 看原文是否支持因果关系，model 放模型和训练组件，benchmark_causal 做独立评测。

- 默认情况下，从命令行 CLI、HTTP、Kafka 或查询入口提交问题，都会交给同一个 AgentService，跑同一套 LangGraph 流程。

- 以前的查询、缓存和五阶段 Worker 还留着，方便旧代码继续用。它们没有全部改成新流程，使用时要区分。

- 流程里有这些节点：L0 网关、Planner、Executor、Replanner、AwaitInput、Generate、WriteCode、ExecuteCode、InjectResult、Cleanup。根据任务情况选择下一步，不是每次全部跑一遍。

- 闲聊直接回一句固定问候；简单问题直接查资料；复杂问题先让 Planner 拆成小任务。

- 可以按 pyproject.toml 安装成 legal-rag-agent 包，再用 legal-agent 命令运行。旧模块的导入和运行入口也保留了。

- 项目运行文件、脚本缓存和模型缓存都按约定放在 D 盘。

任务怎么拆、怎么执行、失败后怎么办

代码在这里：双层计划、operators、nodes、scheduling、硬规则。

- L0 会用中英文关键词和特殊分隔符，检查一部分提示注入写法。这只是规则判断，不能识别所有情况。

- 先用同一套规则把问题分成闲聊、简单问答和复杂任务，这一步不用调用模型。

- Planner 可以给两层计划：S_q 是要做哪些事、谁依赖谁的 DAG；C_q 是把这些事具体写成检索任务。

- 系统会检查依赖有没有写错、有没有循环，排出执行顺序，找出能同时做的任务，再转成任务队列。

- 旧的扁平 task_queue、strategy_queue、字符串任务和 [WORMHOLE] 标记也能继续解析。

- 每个 TaskSpec 都记任务 ID、要做的事、依赖、检索方式、理由、证据处理方式和计划版本 revision。

- 简单问题不用再让 Planner 规划，直接查资料，而且这个直接任务默认不做因果分析。

- 每个任务会先查资料，再让 Grader 提取事实、判断信息够不够；需要时单独推理，之后记录过程并更新完成列表。

- Grader 会给出信息足够 sufficient、还缺一部分 partial、资料不相关 irrelevant 三种结果。事实、判断理由、缺什么和缺哪类信息都会分开记。

- 如果缺的是案件事实，就问你；如果缺的是资料，就重新检索或调整计划。

- 每个任务自己保存失败原因和重试记录。失败达到阈值，或者资料不相关时，再交给 Replanner。

- Replanner 可以换检索词、改任务或换检索方式。它只改失败的分支，不让已经成功的兄弟任务重做。

- “虫洞”就是跳出当前语义图附近，重新做一次全局向量检索；资料还是同一套，不是又建一个数据库。
- 重新规划会提高任务版本 revision，用来分清旧计划和新计划的结果。
- 递归深度和运行预算会限制反复检索、重规划和执行，避免一直转圈。
- 多个任务的结果合并后，还会按规则查有没有漏项。最后一个任务做完也会查。
- 规则补出的任务有固定 ID，会去重，并按规定的依赖顺序执行。
- 目前有七类补漏规则：未签劳动合同的二倍工资、工伤认定及赔偿、社保缴纳和金额、加班费、竞业限制和补偿、试用期合法性和解除条件、劳务派遣责任及法律关系。
- replanner_rules_report.py 里还有扩展场景分析，但那些不等于已经加入运行规则。
- 任务完成后会存下事实、计划版本和用户补充。后面的依赖任务可以拿到前面查出的事实。
- 你补充的信息只给对应任务，以及明确依赖它的任务，不会自动发给其他无关分支。
- 每次观察结果都会追加保存；事实和评分理由分开。最后由 Generator 汇总这些记录和已经读过的证据。
- 常规任务由 Grader 提取事实，不是每一步都先跑 Extractor、再跑 Reasoner。只有任务要求推理、而且资料够了，才单独调用 Reasoner。

哪些任务能同时做，资源怎么分

代码在这里：scheduling、resources、共享编码器、并行说明。

- 前置任务已经完成的小任务，可以选出一批放到线程池里同时执行；一批最多几个，由配置决定。
- 开跑前会检查任务 ID 是否重复、依赖是否存在，以及有没有互相等待的循环。
- 每个分支各自记运行信息、重试和检索过程，但同一次任务共用一份只读知识资料。
- 分支做完后，由一个协调器按计划顺序合并结果。谁先完成，不会改变事实的排列顺序。
- 分支结果会先存好。其他分支失败或进程退出后，已经完成的部分可以继续用。
- 同时运行几个问题、几个子任务、几个远程 / 本地模型请求，以及几次编码和沙箱执行，都可以分别限制。
- 排队等待的数量也有上限。等待时仍会检查任务有没有取消、是否超时、执行者是否还有权限继续。
- 同一台机器上的不同进程通过文件锁协调资源，进程退出后会释放锁。
- 本地 GPU 模型有一个进程负责持有。第二个进程不能再重复加载一份常驻权重。
- 同一进程会复用本地 vLLM。要换已经加载的模型配置，需要重启持有模型的进程。
- 模型、设备、精度、维度和嵌入空间相同的编码器可以复用，最多缓存几份也有限制。
- 对同一个编码器的调用会排队执行。自己传入的编码器仍保留自己的身份，不会被误认成另一份模型。
- 会记下等资源花了多久、工具执行花了多久，方便之后找慢的地方。
- 目前是一批做完再开下一批。还没有抢占任务、严格公平排队、多机调度、按显存字节分配资源或自动安排多张 GPU。
- 这里借鉴了 TokenSpeed 把计划、执行和完成分开管理，以及限制资源并发的思路，没有直接引入它的推理引擎或内核库。

怎么算金额，怎么运行生成的代码

代码在这里：calculator、节点、sandbox、后端说明。

13.1 默认怎么计算金额

- 金额计算先让模型给出有限数值、公式、步骤、结果变量和假设，再由程序做加减乘除。
- 开算前会检查数值是否有限、大小是否超范围、变量名和步骤数是否合适，以及有没有引用未算出的值或重复赋值。
- 计算用 Decimal 和固定精度，金额按 HALF_UP 四舍五入到两位小数。
- 算错就明确报失败。算出的结果、用的公式和假设都会留在报告里。
- 计算结果也会存进工具记录，恢复时可以直接用。先完成程序核算，再生成最后的报告。
- 程序能检查怎么算，但不能单靠算式证明法律用得对、输入事实是真的。

13.2 需要运行代码时，怎么用 Docker

- 只有设置 allow_code=true，统一运行时才会让模型生成 Python 代码执行。
- 代码出错时，可以带着错误让模型修改，但重试次数有限。如果是配置错误或沙箱不可用，就不会反复改代码。
- 次数用完仍没算成，会记录计算失败并停止，不会把错误提示写成“已经核算成功”。
- 执行结果会统一带回计算值、错误、会话标识、用了哪个后端，以及执行和清理的状态。
- 默认用 Docker 跑代码：断网、非 root 用户、根文件系统只读，并限制 CPU、内存、PID 数量和输出大小。
- 代码通过 Docker exec 传进去，默认不额外开放沙箱 HTTP 端口。
- 每次都启动新的 Python 进程。优先取代码里的 result，没有就看 stdout；上次的变量不会留到下次。
- 可以创建、执行、超时终止和销毁沙箱，也有空闲 / 孤儿清理方法。清理要主动调用，或由已有的执行结束流程触发。
- 执行后和整个任务结束时都会尝试清理。没清干净就保留标识和错误，方便后面继续处理。
- run_in_process 只是手动传入、供可信离线测试使用的函数，没有隔离能力；线上不会自动退回宿主机执行。
- 旧 FastAPI 沙箱服务和包装文件还在，但默认在线执行用 Docker exec。

13.3 需要时怎么换成 SPACE

- SPACE 默认关闭。当前只适配固定上游版本的 native /v1 M1 接口，用来一次性执行 Python。
- 能创建沙箱、查 operation 进度、等沙箱和进程服务准备好、创建命令、读 SSE 结果，再删除沙箱并等删除完成。
- 管理接口的 Bearer Token 和事件流的 Session Token 分开使用。秘密凭据不会写进日志或请求记录。
- 创建、执行和删除前，会先存下请求方法、路径、请求体、幂等键和关联 ID，方便后面确认到底做过什么。
- 创建请求发出后没收到响应等情况，可以按记录核对。只有符合允许重放的条件才重发，不盲目再做一次。
- 启用前会检查模板是不是固定摘要版本、资源单位、地址、超时、幂等有效期，以及部署网络隔离的核对记录。
- 连接、等待数据、各阶段操作和整个过程都有时间上限；请求、事件帧、整条流和输出也有大小限制。
- SSE 能处理分片、多行 data、CRLF 和心跳，还会核对收到的退出消息是不是来自这条命令的进程。
- 只有正确进程正常退出、退出码为 0，而且 runner 返回合法 JSON，才算执行成功。
- 断流、错误帧、resync、没有退出事件、异常退出或结果格式不对，都会记为失败。
- 一次性会话不能继续跑第二条命令。如果恢复时已经有可用的缓存结果，就直接返回，不重新连接远程后端。

- 网络和虚拟机隔离还得靠真实部署与模板来保证，写了适配代码不等于已经验证隔离有效。
- 目前不能暂停 / 恢复虚拟机、做快照、用预热池、传文件或共用跨任务工作区，也没做 E2B 兼容层。

明细五：流程变成可靠服务

任务从提交到结束怎么管理

代码在这里：contracts、service、store、HTTP API。

- 每次提交问题都会得到一个单独的 run_id。系统会记下项目 ID、输入参数、提交方式和可用预算。
- 提交时可以带上问题、项目 ID、预测截止日、目标实体、运行预算，以及是否允许执行代码的 allow_code。
- 空问题、太长的问题、不合法的项目 ID、错误日期，以及系统不认识的额外字段，都会直接报错。
- 任务有七种状态：排队 queued、运行中 running、等你补信息 waiting_input、完成 succeeded、失败 failed、取消 cancelled、超时 timed_out。
- 可以查一次任务的状态、结果、错误类型、预算花了多少和其他运行信息，也可以按项目看最近的任务。
- 运行过程中发生的事会存下来。读完一批事件后，可以从上次的位置继续查新的事件。
- 你可以取消任务。系统用租约和 epoch（执行轮次）拦住旧执行者，避免它在取消后又把结果写回来。
- 缺少只有你能提供的事实时，任务会停下来，把问题存好；你回答后，它会从保存的位置接着做。
- 补充答案不能为空，也不能超过长度限制。等待和继续任务都沿用原来的截止时间 deadline 和预算，不会重新计时。
- 失败后可以主动重试；排队任务会继续执行；执行者失联、租约过期后，也可以扫描出需要恢复的任务。
- 错误会分开记：预算用完、资源冲突、模型连接问题、输入或返回格式不对、结果无法确定、失去执行权，以及程序内部错误。
- 只有流程没有剩余节点、已经生成报告，并且没有预算耗尽或计算失败标记，才会记为完成。
- succeeded 只表示程序跑完了。法律结论对不对、事情有没有真正办好，还要另外判断。
- HTTP 任务接口使用团队共享的 Bearer Token。除了 /ready，其他任务接口都要带这个令牌。
- HTTP /ready 会看运行数据库能不能读、本地执行线程还在不在。它不会替你检查外部模型的回答质量。
- HTTP 本地模式会在后台执行任务；Kafka 模式先把任务存好，再交给同一台机器上的 Worker 去做。
- HTTP 会用不同状态码说明问题：找不到任务是 404，状态冲突或结果无法确定是 409，输入不对是 422，服务没准备好是 503。

任务怎么保存、恢复、回滚和备份

代码在这里：store、检查点写入、执行上下文、备份恢复。

- SQLite 运行库会存任务状态、租约、epoch、预算、事件、工具调用记录、本次任务固定使用的资料，以及等着发出的消息 outbox。
- 旁边还有一个 SQLite 检查点库，保存 LangGraph 跑到了哪里。进程退出后，可以靠它接着跑。
- 执行者要定期续租，证明自己还在工作。同一次任务只认一个有效执行者。
- 写检查点或新增状态前，系统会确认写入的人还有执行权，避免失联的旧进程改坏新状态。

- 任务第一次读知识库时，会固定一份图快照和知识版本 revision。恢复后仍用这份资料，不会中途换版本。
- 模型配置、提示词和已启用的工具命名文件都会记下内容哈希。如果它们变了，就不能拿旧检查点直接续跑。
- 工具结果先存入调用记录，再保存节点检查点。恢复时，已经存好的读取结果或分支结果可以直接用，不必重做。
- 每条工具记录会区分运行、epoch、节点、步骤、任务 ID、任务 revision 和输入摘要，避免把不同调用混在一起。
- 工具出错会保留错误。沙箱里涉及创建、执行或删除的请求也能先记下来，结果不明时再核对。
- 可以查看当前任务的检查点列表，选一个以前的位置退回去。
- 退回后会提高 epoch，相当于开始新一轮执行。旧消息、旧分支和旧执行者不能再覆盖新状态。
- 如果无法确定外部操作有没有发生，会记成 outcome_unknown，不会直接假定它没执行。
- 运行状态和待发消息 outbox 会一起存成功。消息代理 Broker 确认收到了，系统才把消息标为已发送。
- 备份前要先确认没有仍持有有效租约的任务。复制期间也不让新的执行者接任务。
- 备份用 SQLite backup API 复制运行库和检查点库，检查文件完整性，再生成带 SHA-256 校验值的 manifest 清单。
- 恢复时会核对文件哈希和数据库是否完整。只写到新目录，不覆盖现有数据库。
- 备份里包括任务固定使用的证据资料。项目知识库、模型、原始 PDF 和凭据还要另外备份。
- 回滚能退回任务状态、重新使用已存结果，但撤销不了已经产生的模型账单，也不能自动撤销外部系统里的操作。
- 目前不会自动清理运行历史、检查点、快照和备份，保留多久还要另外设置。

Kafka 怎么接，旧链路和部署文件怎么用

代码在这里：transport、Worker、旧链路。

- 新的 Kafka 消息带 version、event_id、run_id 和 epoch。具体任务内容还是以运行数据库为准。
- 待发消息先存 outbox，再发送、等 Broker 确认和核对通知。结果消息只提醒按 ID 取结果。
- 旧 epoch、已结束任务或对不上的通知，都不能重新拿到执行权。收到几次消息，不代表就执行几次。
- Worker 的消费线程持续收消息，执行线程另外扫描任务库。任务很长也不应该卡住消费轮询。
- 新的 pending / result topic 和旧五阶段 topic 分开，不混用协议。
- 旧的 Planner→Retriever→Grader→Replanner / Reasoner 五阶段消息链还保留着。
- 旧 Redis StateManager 仍能读写 JSON 状态、设置 TTL。会话没了或过期就报错；新运行恢复不靠它判断状态。
- 旧 Kafka 工厂统一 producer / consumer 设置，手动提交消费位置，并用会话键决定分区。
- 旧链路处理不了的消息会写本地 DLQ 隔离。如果记录没写成功，就不能当成处理完了去提交消费位置。
- 旧 Retriever 仍能做向量检索、图扩展、文档导航，以及来源证据和 CHAIN 处理。
- 旧 Reasoner 还保留报告生成、金额检测、代码生成、沙箱和有限重试；旧 Replanner 有自己的结构化接口和硬规则。

- 设置 runtime.enabled=false 可以走旧查询方式；容器设置 AGENT_RUNTIME=0 可以启动旧阶段 Worker，用于处理剩余旧任务。

- 仓库里有 Worker Dockerfile、启动 entrypoint、旧 Kubernetes 部署示例，以及 Docker 沙箱镜像文件。

- Kubernetes 示例没有接好新架构要求的运行库和知识库共享存储，不能直接当成新系统的多机部署方案。

- 新 Kafka 模式下，API 和 Worker 必须在同一台机器，使用同一份数据库和资源目录。本地模式不用 Kafka / Redis。

明细六：准备数据和改进模型

怎么调用模型、使用本地模型和缓存

代码在这里：llm、vllm_engine、语义缓存、缓存存储。

- 模型调用统一走 OpenAI 兼容接口，默认用 API。地址、凭据、模型名都能配，也能给不同角色指定不同模型。

- 每个调用用 ContextVar 记自己的角色，避免两个并行任务互相改掉模型角色。

- 提示模板统一放在 prompts.py。运行时只把本次事实填进去，不在各处另存一份模板。

- 要求 JSON 时会解析返回内容。空回答或非法 JSON 会直接报错。

- 模型请求有超时限制，连接等可重试错误只重试有限次数；每试一次都会消耗调用预算。

- 任务剩多少时间，也会限制这次请求能等多久。返回结果存进工具记录，之后恢复可以复用。

- 主接口的 api / auto 都用 API；你指定 vllm，本地启动失败就报错，不会悄悄改用付费 API。

- vLLM 包装可以复用相同前缀的缓存，设置温度和输出长度；目前最大模型长度固定为 8192。

- 同一基座可以挂多个与它匹配的 LoRA，再按角色选。适配器 ID 保持稳定，配置了不存在的名字会报错。

- 目前还不能自动换不同基座，也没有把 4bit 推理、CPU 卸载或张量并行参数接进这套配置。

- 旧 create_llm_backend 工厂还保留自动检测、失败后退回 API 的行为。别把它和现在的主接口混为一谈。

- 旧语义答案缓存可以存在内存向量库或 RedisVL 里。相似问题可以查旧答案，完成的问答也能写回。

- 旧路由可以在开始处理前命中答案缓存。但开启长期记忆或 CHAIN 后会跳过它，因为旧缓存没记录资料版本和时间条件。

- 旧路由还保留规则判断、向量语义分类和 LLM 补判三层。现在默认的持久任务用的是共享规则分类，不先调用模型。

- vLLM 的前缀缓存、旧答案缓存、长期知识和任务检查点各有用途，不是同一份东西。

PA-Tool 怎么把工具接口改成模型更熟悉的名字

代码在这里：schema_alignment、CLI、说明。

- 这个组件可以开关，默认 profiles: {}。不配置时，工具名字和原有行为都不变。

- 目前能改 Replanner 的两个检索引擎名和四个字段，以及文档导航的五个动作名和六个参数名。

- 先让实际使用工具的模型在温度 0 下给参考名字，再按设定温度生成多个候选；也可以自己传模型调用函数。

- 用字符级 Levenshtein 距离看候选名字有多像，再结合出现次数，计算聚集程度 peakedness。

- 距离阈值是候选最大长度乘 alpha。聚集分一样时，选更接近参考名的。

- 不同组件抢同一个名字时，优先级高的先用，其他的往后选。如果和保留字段冲突，或候选不够，就报错。
- 选好的名字存成 profile，记住它对应的角色、后端、基座和适配器配置，同时保留候选和排序。
- 可以读已经保存的候选来生成 profile，也可以主动调用本地目标 vLLM 模型来取候选。
- 主动采样时，必须指定后端，而且本地已经有模型；不会自动下载，也不会训练。
- 静态工具说明和 JSON Schema 会一起改名，字段、required、动作枚举和默认值保持对应。
- 模型返回后，再把新名字翻译回项目原来的字段和动作，然后交给现有检查和执行代码。
- 只改工具接口名字，不全局替换案情、原文、目录、日志或自由文本，也不会猜一个相似工具名就执行。
- profile 内容会算进运行配置哈希。想关闭时，删掉对应角色的配置就行。
- 配置身份只能说明写了哪个模型，不是完整权重指纹。换目标模型后，要重新采样并检查。
- 目前还没用真实目标模型生成映射，也没测出工具成功率提升；旧 Redis 阶段的 Replanner 没接这个组件。

怎么减少本地模型的迎合和套话

代码在这里：behavior、依赖、说明。

- 这是一个独立、可选的离线组件，调用固定 Heretic 2.0.0 的 KeywordRate、KLDivergence 和 ARA。
- 可以选减少指定迎合词、减少指定套话，或两者一起做；词表可以自己换。
- 要先准备完整本地模型，以及 good / bad / evaluation 三份提示文件。组件会检查输入和新的输出目录。
- 默认只保存提示副本、配置和 manifest，不加载大模型，也不启动外部程序。
- 只有加上 --run，才会用指定的独立 Python 执行，核对 Heretic 版本，而且只用本地已有模型。
- 这个子进程的缓存、临时目录、检查点、搜索次数、随机种子和输出长度都单独设置。
- 结果另存为一个候选 LoRA，不覆盖原模型，也不自动上传或启用。
- 会记下提示、配置和模型 config 的哈希，以及执行状态。即使导出成功，质量仍标成未验证。
- 通过现有角色适配器配置就能接上或拔掉。每次请求只能选一个 LoRA，不能自动和已有任务 LoRA 叠加。
- 它不处理最终 Generator API，也不使用默认的拒答抑制目标。
- 套话关键词少了、KL 变化小了，不代表模型一定更理性、更真实或任务做得更好。目前没有跑实际权重优化实验。

怎么生成合成数据

代码在这里：synthetic、Evol-Instruct、说明。

17.1 新的独立合成组件

- 默认可以完全离线使用：不加载模型、不调 API、不训练，也不改在线证据库。
- 可以生成六类任务：单次查询、多记录汇总、依赖链、并行分支和角色交接、工具临时失败后的恢复，以及信息 / 权限 / 预算限制。
- 组件先造一个数值或状态环境，再由程序算出正确目标。需要写入时，也只改当前模拟任务的内存。
- 任务类型比例、实体组数量、每组变体数、记录数和依赖链复杂度都可以配置。
- 复杂度和变体数可以按有上限的幂律采样，让少量任务更难；这不代表已经拟合了真实业务的长尾分布。

- 可以加同文复述、过期版本、低优先级冲突和同级未决冲突。改了任务后，会重新计算目标。

- 来源优先级和 active 标记只是事先定义的模拟规则，不代表现实里的法律效力。

- 可以加干扰工具、改工具名、改 JSON 返回字段、改问题说法，用来测这些结构变化。

- 先按稳定的组 ID 分 train / validation / test，再生成同组变体；OOD 另用不同实体组，避免把同一组换个说法就当新测试。

- 可以拿你提供的种子资料生成待审题目。共享来源 ID 或去掉首尾空白后相同的文本，必须留在同一来源组。

- 给求解者看的任务输入，和完整环境、正确标签分开保存，避免把答案跟问题一起喂给模型。

- 数值和状态任务生成后，会真正回放一次参考步骤再发布。不过，这只验证模拟环境里的规则。

- 法律、因果和开放推理题会放到 pending 待审区，可以带拟定答案 proposed answer，但不能自动变成 verified。

- 可以接 CandidateProvider，把生成请求和 Schema 交给自己的回调。只有明确传入它、而且调用额度大于零，才会使用。

- provider 出错就停止报错，不自己重试或换模型。额度用完后改用事先安排的离线模板，并明确标出来源。

- 可选生成几种轨迹：程序参考、有效但多余、伪造观察、调用非法工具、最终答案错误。

- 这些轨迹标成 scripted_reference，是程序写出来的样例，不是模型实际跑出来的 rollout，也不是真实 Agent 日志。

- 评分分开看答案对不对、步骤是否合法、最终状态对不对，再扣固定工具调用成本，并记录 reward version。

- 同一任务里，两条轨迹分数不同才组成排序对。目前没写评分模型损失，也不会自动开始 RL 训练。

- 训练 / 验证 / 测试分别输出 verified 和 pending；需要轨迹时再输出 trajectories 和 ranking，另外还会统计各组曝光次数。

- 曝光统计会区分“来源放进环境几次”和“参考轨迹实际读了几次”，它不是模型训练时见过多少 tokens。

- manifest 会记录配置、运行版本、代码和文件哈希、实际采样分布，以及 provider 调用了几次。

- 一组组生成、一条条写出，先写临时目录，全部完成后再发布，不覆盖旧输出。

- 生成任务都会标 synthetic；种子资料单独标明来源，不把合成内容装成实际证据。

17.2 以前的 Evol-Instruct 生成器

- 旧生成器可以给定一段法条，让模型反过来造案件问题、两层 Planner 计划、关键事实和计算代码候选。

- 也可以从判决文字生成口语化、有噪声或故意缺信息的问题。

- 还可以从案件集合生成失败和重规划场景，输出 JSONL。

- 它使用集中的提示模板和 AsyncOpenAI。不过，部分批量逻辑仍是一条条请求，不是高并发生成服务。

- 旧代码虽然把产物叫 Ground Truth，但没有因此多出程序验证或法律审核，也不会自动放进新组件的 verified 集合。

- 旧模块导入时就会创建客户端；有些请求失败后会记日志、跳过或返回空列表，处理方式和新合成组件不同。

现在有哪些训练代码

代码在这里：训练目录、QLoRA 加载器、训练依赖。

训练什么	现在怎么写的	读什么、输出什么	还要注意什么
Planner	Qwen2.5-14B，用 QLoRA 做 SFT，先用 Semantic DTW 筛样本	读 planner_trajectories.jsonl，存 Planner LoRA	DTW 只能看语义对齐，不能证明任务做成了；旧筛选代码还有异常兜底
Replanner	Qwen2.5-14B，用 GRPO，候选按 8→4→2 打分	读 replanner_scenarios.jsonl，存 Replanner LoRA	目前靠模拟检索、相关性和关键词 / 虫洞奖励，没有完整执行真实任务来验收
Extractor / Grader	Qwen2.5-3B，用 chosen / rejected 对做 DPO	读 extractor_pairs.jsonl，存 DPO LoRA	学会偏好某种文字回答，不代表已经学会按现在的 Grade 格式输出
Reasoner	Qwen2.5-7B，用长上下文 IRAC 数据做 SFT	读 reasoner_qa.jsonl，存 SFT LoRA	有脚本和示例答案，不代表法律结论已经验证正确
Retriever	BGE-M3，在 q / v 挂 LoRA，用 MultipleNegativesRankingLoss	读 retriever_triplets.jsonl，存 SentenceTransformer 产物	模块名、模型格式和依赖版本能否配合，还要实际训练检查

- 加载器默认用 4bit，可以另外指定模型和序列长度，也支持梯度检查点和 Chat Template 对话格式。

- LoRA 挂在 q / k / v / o / gate / up / down 这些线性层上；不同角色各有自己的 rank、alpha 和上下文长度。

- 适配器和 tokenizer 单独保存，同一个基座不必为每个角色再存一整份。

- 脚本里有 batch size、梯度累积、epoch、学习率、日志和检查点设置。不过，部分值直接写在脚本里，改 YAML 不一定会跟着变。

- Planner 用嵌入相似度矩阵和 DTW，比较计划步骤与关键事实能否对上；两层计划和旧扁平计划都能解析。

- Replanner 可以用 CrossEncoder 打分，缺依赖时旧代码会用词重合。最后的奖励还不是“把真实任务完整做完后，由强模型或程序验收”的结果。

- 部分训练脚本找不到数据文件时，会使用内置示例。能跑示例，不代表正式训练数据已经准备好。

- 部分训练脚本一导入就会加载模型或训练，不能当普通服务模块随便导入。model/Unsloth.py 只演示加载基座、挂 LoRA，不训练。

- 目前没有实际训练、GPU 容量 / 速度测试，或训练后业务效果的验证结果。

- 还没做 FSDP / ZeRO 多卡分片、CPU / NVMe 训练卸载，也不能自动按顺序训练所有模型或部署多个基座。

明细七：检查、配置和使用方式

怎么评测检索、执行过程和因果推理

代码在这里：benchmark、benchmark_causal。

19.1 检索和执行过程怎么打分

- 评测时可以传入自己的检索器和 Agent runner，按同一套格式提供题目、召回结果和执行轨迹。

- 每个问题只检索一次，拿最大 K 的结果计算 HR@K、首个相关结果的 MRR 和噪声比例。

- Graph-NIAH 会看多跳需要的证据找回了多少、有没有全部找齐。找齐节点不等于证明 Agent 真的沿正确路径走过。

- 可以让模型裁判看计划、执行步骤、重试和最终答案，给轨迹打分，再算平均分。

- 有完整评测入口、示例题和模拟检索 / 执行器。要评项目实际效果，还得换成真实 runner。

- 旧裁判请求失败时返回 ERROR，返回内容不是有效分数时按零分处理。所以，代码里有指标不等于已经有可信业务结果。

19.2 用 SCM 做可计算的因果测试

- SCM 里要明确写出外生 / 内生变量、依赖边、结构方程、来源法条和可用数值范围。
- 它会检查变量、方程和依赖是否合理、DAG 有没有环，也能查执行顺序、父子节点、祖先后代和混杂结构。
- 可以按当前条件算结果、做 do 干预、问反事实，也能查看因果链、结果差异和图说明。
- 自带加班费 / 夜班津贴和违法解除赔偿两个场景，分别覆盖有混杂和无混杂的结构。
- 同一个场景可以生成 L1 观察、L2 干预、L3 反事实和加无关信息的版本，并保留共同来源 ID。
- 参考答案由场景里的结构方程计算，法条是明确指定的依据。不过，方程本身也是人工设定的假设。
- 法条库读 JSONL，统一法名和中文条号，支持同一法条的多个版本、按日期检查有效性，以及核对引用。
- 也能从本地已有的 LawRefBook 风格 SQLite / Markdown 仓库提取选定法条，保存来源和版本信息；不会自动下载仓库。
- 可以从模型文字里取出完整 JSON，再检查结论、因果链、引用和金额这些字段。
- 先过几道硬检查：格式对不对、法条存不存在、当时是否有效、关键结论和数字是否对上、金额误差是否允许。没过就零分。
- 推理打分可以自己传 matcher，默认按词语和数值匹配。如果混杂判断缺失或本来不适用，就不硬算这一项。
- 会输出每道题的分数和理由、各层平均分 / 通过率，以及同一来源题组是否都通过。
- 目前只对实际传入的层打分，不强制要求 L1 / L2 / L3 都在。同层的扰动版本还可能互相覆盖，所以还不能说完整测了鲁棒性。
- SCM 只用于离线评测，没有当成在线工具，也不会直接把 CHAIN 的文本图变成结构方程。

日志、指标、配置和检查工具

代码在这里：observability、config_loader、CI、入口审计。

21.1 怎么看日志和运行情况

- 日志可以用 JSON，也可以用普通文本，会记时间、级别、模块、函数、行号、异常和额外字段。
- 可以在线程安全的收集器里记 Counter 次数、Gauge 当前值和 Histogram 样本，再看 count / p50 / p95 / p99 / min / max。
- 旧的独立 HTTP 健康服务提供 /health、/ready 和 JSON /metrics。这个 metrics 不是 Prometheus 文本格式。
- 这个旧服务的 ready 只是简单状态，不像运行 API 那样检查实际运行条件，也不代表模型已经可用。
- 有可复用的指数退避重试工具，以及独立主备 API 客户端，会记调用时间和失败次数。
- 主备客户端是旧的独立组件，默认新模型接口不会自动采用它的回退逻辑。
- 本地 JSONL DeadLetterQueue 可以写失败消息、查看和回放，但它不是 Kafka DLQ Topic 服务。
- LLMTimer 可以给一段模型调用计时；任务事件、工具记录、资源等待和导航 trace 也能用来排查问题。

21.2 两个默认关闭的性能组件

- 批量编码会把长度接近的文本放一组，限制每批数量，保留 sparse，归一化 dense，最后恢复原来的顺序。

- 这条可选路径能用于文本向量化、没有预先向量的初始建图，以及 PDF 原页导入；没有输入时不加载模型。

- 原来逐条编码的代码还在。自己传入单条编码函数时，不会擅自把它改成批量接口。

- 快照索引缓存把 CPU FAISS 索引保存成当前进程里的字节。缓存键会包含完整快照和项目分区的哈希。

- 缓存用 LRU 淘汰旧内容，同时限制份数和字节数。第一次建索引时加锁，避免同一份快照重复构建。

- 命中缓存后，会还原出本次任务自己的索引，不共享可修改的图。索引太大也能临时用，只是不留在缓存里。

- 批量编码和索引缓存默认都关着。这里没有自定义 GPU 算子、零拷贝或跨进程缓存。

- 缓存大小限制只管它留住的字节，不包括正在运行的图、索引和临时内存；批量编码也不限制最后整套语料有多大。

- 会记录语料编码、批量编码、准备知识视图和索引缓存各花多少实际时间，不是用 CUDA Event 测单个算子。

- 目前还没有真实的数值一致性对照和性能收益结果。

21.3 配置、依赖和检查工具

- 配置统一从一个 YAML 读取，可以用 MY_RAG_CONFIG 换文件，也能用环境变量替换值、设置默认值和按层取参数。

- 文件不存在、YAML 写错或顶层结构不对，会直接报错。缺凭据会告警，也可以主动要求严格检查。

- .env.example 只是说明要设置哪些变量，项目不会自动加载 .env。

- 配置里包括模型、向量、图、记忆、导航、因果、消息、缓存、沙箱、调度、部署预算、日志和训练参数声明。

- 路径检查和 tools/run.ps1 会要求项目留在 D 盘，并把临时文件、包缓存、模型缓存和 UTF-8 设置好。

- 运行、开发、训练、资料检查和行为优化各有依赖文件。runtime constraints 只约束部分已有依赖，不是完整环境锁。

- 打包配置列出了包、模块、命令行入口和规则 JSON，支持 editable 安装和 wheel 构建。

- .gitignore 会跳过凭据、运行数据库、缓存、模型权重和生成训练数据等本地产物。

- 入口检查会在单独子进程里试导入清单中的模块；它没有覆盖所有训练脚本的顶层行为。

- CI 会做五类检查：入口导入、compileall 语法编译、pytest、离线 demo，以及跨进程检查点恢复 spike。

- 有七个离线 demo：任务保存与恢复、语义图、记忆桥接、增量阈值、LangGraph 完整流程、因果沙盘和沙箱返回结果。

- spike 会演示 interrupt 之前的节点可能重跑、检查点如何保存、另一个进程怎么 resume，以及如何靠外部记录避免重复派发。

- 测试里可以用固定返回的模型、编码器、消息和沙箱替身，还有专门跑图状态的 harness。

- 这份清单整理时只读代码和文档，没有重新跑业务测试或训练。后面的测试文件和记录是查证入口。

- 项目没有单独配置固定的 lint 和类型检查流程；compileall 只能检查语法编译，不能当类型检查。

- training_data_guide.py、旧 README、规则报告和简历提醒都是说明或研究记录，不是程序会执行的业务能力。

现在还做不到什么

前面聊过的想法，并不是都已经写进项目。下面这些容易混淆，所以单独列清楚：

- 知识语义图、文档目录、文本因果超图和离线 SCM 各管一件事，没有合成一种图。另外还有任务 DAG，它安排做事顺序，不存证据。
- 不会把同一原文自动复制成四份常驻 GPU 数据库。不过，CPU 向量副本、图快照和任务快照仍会占内存和磁盘。
- 没有用 GNN 学因果图，没有统计估计真实因果效应，也没有完整的 CHAIN 概率预测系统。
- 没有整套 PageIndex flash、按成本自动展开 / 合并树、完整供应商集成或它的云服务。
- 还不能完整处理 OCR、复杂法律表格和版面，也没全量接入 AnyDoc / Firecrawl 或搭建自动人工复核平台。
- 还没有 JEV 小模型决策、CASCADE 阈值拟合、Shadow Mode、工作流搜索、MASS 轨迹蒸馏或 Terminal-RSI 训练。
- 没有在线自动改代码、自动训练和发布、训练失败回滚，或自动批准模型上线的流程。
- 没有自写 Triton / CUDA 算子，也没移植 TokenSpeed 推理引擎，更没有对应的硬件吞吐收益实验。
- 模型训练分片、显存流式加载、不同基座热切换和多个 LoRA 自动叠加都还没做。
- 还没有多租户账号权限、项目级 ACL、分布式存储、多机高可用、严格公平 / 抢占调度或分布式 exactly-once。
- 没有一套能撤销任意外部写入的通用系统。沙箱的创建、执行和删除也会影响外部服务，需要记下并核对。
- 退回检查点不等于撤销外部操作；换了 epoch，也不能据此断定前一次请求没生效。
- 还没有真实模型质量、提取 / 校准收益、检索收益、训练收益和 GPU 性能的完整验证结果。
- 以前的离线测试通过，不代表已经测过真实 Kafka / Redis / Docker / SPACE，也不代表干净环境能完整安装部署。
- 面试时要提醒你的事，单独放在 RESUME_REMINDERS，不会自动写进简历。这次没有改那份提醒。

默认开了哪些功能，限制是多少

这些值来自 config.yaml、Budgets 和模型接口，表示现在代码怎么设，不代表已经调到了最合适的值。

设置	现在的值	用来做什么
runtime.enabled	true	默认保存任务状态，支持继续执行
llm.backend	api	默认外部 API
llm.role_models	空映射	默认角色共用配置模型
embedding.device	cuda	嵌入默认用 GPU；想用 CPU 要改配置
long_term_memory.enabled	true	可以读取以前保存的知识
知识库	.memory/rag.sqlite	项目知识快照
运行库	.runtime/runs.sqlite	任务状态和执行记录以这里为准
检查点库	配对 runs-checkpoints.sqlite	LangGraph 检查点

设置	现在的值	用来做什么
pageindex.enabled / navigation	true / auto	自动选择短文直接读或导航
导航轮数 / 原文记录数	4 / 8	最多导航几轮、读几条原文
原文 / 目录 tokens	6000 / 3000	阅读与目录内容预算
causal_evidence.enabled	true	可以用因果组件，具体任务再决定要不要用
校准文件 / 默认截止日 / 目标	null / null / null	还没指定校准文件、预测日期和目标
CTVF d_max / eta / 缺失时间有效性	3 / 5.0 / 0.25	权重参数
因果返回链 / 扩展 / 最低分	200 / 10000 / 0.005	搜索预算
allow_code	false	默认让 Decimal 按计划算，不执行生成代码
沙箱后端 / SPACE enabled	docker / false	代码模式默认 Docker
Docker 内存 / 超时 / 输出	512m / 10 秒 / 65536 字节	代码执行资源限制
沙箱最大重试	3	限制改代码和重试次数
并发运行 / 并发任务	2 / 2	同时跑几个问题、几条分支
远程 / 本地模型 slots	4 / 1	同时允许几个模型请求
编码器 / 沙箱 slots	1 / 1	同时允许几次编码和代码执行
等待者 / 模型缓存条目	64 / 2	队列与模型缓存数量上限
运行默认预算	300 秒、40 次模型请求、80 节点步骤、100000 文本 token 上界	继续任务时不重新给预算
模型超时 / 额外传输重试	30 秒 / 1 次	还受运行剩余时间限制
最大递归深度 / 虫洞阈值	5 / 2	控制任务反复重试
批量编码 / 快照索引缓存	false / false	默认仍用原来的代码路径
编码批量 / 缓存条目 / 缓存字节	32 / 2 / 268435456	可选性能配置
PA-Tool profiles	空映射	默认不改名
Heretic 行为优化	手动执行，默认不运行	候选适配器不会自动启用

这里的“文本 tokens”用 UTF-8 输入字节数加预留输出估算，算得偏保守，不是供应商实际计费数。slots 只限制同时跑几个请求，不保证显存一定够；配置里有某个参数，也要代码真的读了才生效。

可以用哪些命令和接口

下面列的是现在能用的命令，这次没有逐个运行。具体参数可以看各模块的 --help 和对应说明。

入口	全部已有子命令 / 用途

入口	全部已有子命令 / 用途
legal-agent / python -m agent_runtime	run、status、events、cancel、resume、retry、checkpoints、rewind、recover、backup、restore、serve
python -m agent_runtime.worker	收 Kafka 通知、发待发消息、继续执行已保存的任务
tools/run.ps1	ci、demo、serve、worker
tools/long_term_memory.py	route-corpus、inspect、documents、structure、pages、cost、import-pageindex、import-document
dataset/chunk.py	PDF 原生文本分块、预览
dataset/prepare_corpus.py	文本语料扫描、分块、向量化与导出
tools/calibrate_causal_support.py	按来源分组拟合校准曲线，输出留出指标和校准文件
python -m model.data.synthetic	配置、种子、可选轨迹的离线合成导出
model/data/evol_instruct.py	旧模型数据生成示例入口
python -m model.tool_schema	候选采样 / 读取、工具命名 profile 生成
python -m model.behavior	准备行为优化配置，手动选择是否启动外部程序
model/training/train_*.py	五个独立训练脚本，不由服务启动自动运行
model/Unsloth.py	QLoRA 加载与挂载示例，不训练
benchmark.py	传入自己的检索器 / 执行器来评测，也有示例
benchmark_causal/build_corpus.py	从已存在法条仓库构建评测语料
benchmark_causal/run_demo.py	确定性 SCM 评测演示
tools/audit_entrypoints.py	检查清单中的模块能否按要求导入
tools/ci.py	五类统一离线工程检查
spikes/spike_checkpoint_resume.py	中断、派发去重、跨进程恢复验证

HTTP 的全部任务接口：

方法	路径	用途
GET	/ready	就绪检查
POST	/runs	提交运行，返回 202
GET	/runs	项目运行列表
GET	/runs/{run_id}	状态和结果
GET	/runs/{run_id}/events	游标事件读取
POST	/runs/{run_id}/cancel	取消

方法	路径	用途
POST	/runs/{run_id}/resume	补充事实后排队继续
GET	/runs/{run_id}/checkpoints	检查点列表
POST	/runs/{run_id}/rewind	选择检查点回退

FastAPI 自带 OpenAPI 和交互式接口说明。命令行里的 retry、备份和恢复，目前没有对应的任务 HTTP 接口；也没有任务 WebSocket、SSE 答案推流或浏览器界面。

25. 每个源码文件放了什么

下面列出全部 124 个非测试 Python 文件，以及文件里的顶层类和非私有函数。旧组件、演示和兼容文件也都在；有这个函数，不代表默认流程每次都会调用。内部方法做什么，前面的功能说明已经包含了。只想了解功能的话，可以先跳过这张代码表，需要找实现时再回来查。

文件	主要名字 / 用途
agent_runtime/__init__.py	包初始化、导出或命名空间
agent_runtime/__main__.py	main
agent_runtime/api.py	ResumeInput、RewindInput、create_app
agent_runtime/backup.py	restore_bundle
agent_runtime/checkpointer.py	FencedSqliteSaver
agent_runtime/context.py	record_effect_intent、RunContext、task_context、tool_call
agent_runtime/contracts.py	local_path、RunStatus、Budgets、RunRequest、EvidencePolicy、TaskSpec、Grade、RunConflict、RunStopped、BudgetExceeded、OutcomeUnknown
agent_runtime/demo.py	State、workflow、main
agent_runtime/models.py	SharedEncoder、shared_encoder
agent_runtime/resources.py	SchedulerSettings、settings、ResourceManager、manager、resource、accelerator_owner
agent_runtime/service.py	sandbox_policy_identity、AgentService
agent_runtime/store.py	dumps、RunStore
agent_runtime/transport.py	Envelope、KafkaTransport
agent_runtime/worker.py	main
agent_tools/__init__.py	包初始化、导出或命名空间
agent_tools/calculator.py	Step、CalculationPlan、calculate
agent_tools/sandbox/__init__.py	包初始化、导出或命名空间

文件	主要名字 / 用途
agent_tools/sandbox/errors.py	SandboxError、SandboxConfigError、SandboxUnavailableError、SandboxRemoteError、SandboxProtocolError、SandboxTimeoutError、SandboxOutcomeUnknown、SandboxCleanupError
agent_tools/sandbox/executor.py	run_in_process、run_code_once、destroy_session
agent_tools/sandbox/manager.py	DockerSandboxManager、sandbox_backend、create_sandbox_manager、get_sandbox_manager、reset_sandbox_manager
agent_tools/sandbox/runner.py	CappedOutput、calculate
agent_tools/sandbox/server.py	CodeRequest、execute_code
agent_tools/sandbox/space_client.py	validate_id、SpaceConfig、HttpResponse、UrllibTransport、SpaceEventStream、SpaceClient
agent_tools/sandbox/space_manager.py	SpaceSandboxManager
agent_tools/sandbox/space_runner.py	SPACE 客体执行 runner 源码与版本标识
agent_tools/schema_alignment.py	schema_for、model_identity、edit_distance、rank_candidates、sample_names、build_profile、SchemaAlignment、profile_paths、for_role、profile_hashes
asynchronization/kafka_utils.py	create_producer、create_consumer、quarantine_message
asynchronization/state_manager.py	StateManager
asynchronization/workers/grader_worker.py	call_grader_llm、prepare_grader_evidence、main
asynchronization/workers/planner_worker.py	call_planner_llm、main
asynchronization/workers/reasoner_worker.py	call_reasoner_llm、call_generator_llm、detect_calculation_needed、call_code_generator_llm、run_calc_with_sandbox、main
asynchronization/workers/replanner_worker.py	ReplanTask、ReplanOutput、ReplannerOps、main
asynchronization/workers/retriever_worker.py	build_engine、retrieve_docs、retrieve_hits、main
benchmark.py	llm_judge、TestSample、RetrievalResult、TrajectoryLog、LegalBenchmark、mock_retrieve、mock_run_agent、generate_test_samples、generate_niah_samples
benchmark_causal/__init__.py	包初始化、导出或命名空间
benchmark_causal/build_corpus.py	cn_to_int、parse_markdown_law、build、main
benchmark_causal/gates.py	normalize_text、extract_numbers、ReasoningMatcher、LexicalMatcher、run_hard_gates、ScoringWeights、score_case、aggregate_by_case
benchmark_causal/generator.py	CausalScenario、CausalCaseGenerator
benchmark_causal/legal_corpus.py	cn_to_int、normalize_law_name、parse_citation、LawArticle、CitationCheck、LegalCorpus

文件	主要名字 / 用途
benchmark_causal/run_demo.py	main
benchmark_causal/scenarios.py	scenarios_for、scenario_by_id
benchmark_causal/schemas.py	CausalLevel、LawRef、LegalCausalTestCase、ModelResponse、GateResult、JudgeVerdict、CaseScore
benchmark_causal/scm.py	SCMValidationError、Variable、Edge、Equation、StructuralCausalModel
benchmark_causal/scm_labor.py	build_overtime_scm、build_unlawful_dismissal_scm、get_scm、all_article_refs
causal_evidence/__init__.py	包初始化、导出或命名空间
causal_evidence/calibration.py	SupportLabel、SupportCalibrator、calibration_metrics、fit_calibration
causal_evidence/chunking.py	parse_timestamp、admissible、chunk_record
causal_evidence/extraction.py	canonical、extract_verified
causal_evidence/graph.py	build_graph、active_graph、reverse_distances、score_graph
causal_evidence/models.py	Contract、SourceLocation、EvidenceRecord、TextChunk、Entity、Proposition、CausalRelation、Extraction、Verdict、Verification、Hyperedge、CausalEdge、Rejection、CausalTemporalGraph
causal_evidence/pipeline.py	EvidenceSettings、evidence_from_hit、causal_completion、analyze_evidence、analyze_configured、extraction_identity
config_loader.py	ConfigError、Config
dataset/IncrementalMemoryManager.py	MockGraphDB、DynamicThresholder、IncrementalMemoryManager
dataset/__init__.py	包初始化、导出或命名空间
dataset/chunk.py	extract_text_from_pdf、chunk_text
dataset/citations.py	Citation、CitedAnswer、evidence_catalog、requires_citations、generate_cited_answer
dataset/document_index.py	build_pdf_tree、read_pdf_structure、diagnose_cost
dataset/document_navigation.py	NavigationSettings、Action、NavigationResult、retrieve
dataset/document_store.py	DocumentReader
dataset/encoding.py	encode_batches
dataset/graph.py	vname、as_num_id、LegalDenseGraphBuilder、StubEncoder、make_offline_engine
dataset/index_cache.py	SnapshotIndexCache
dataset/long_term_memory.py	MemoryConflict、MemoryStore、graph_snapshot、rebuild_index、restore_graph、source_evidence_hits
dataset/material_routing.py	MaterialFacts、route_material、route_record、route_corpus、write_routing_report
dataset/memory_graph_bridge.py	MemoryGraphBridge

文件	主要名字 / 用途
dataset/page_index_memory.py	Section、import_tree、import_pdf
dataset/pdf_intake.py	inspect_with_pdf_inspector、import_document
dataset/prepare_corpus.py	discover_files、build_nodes、vectorize、vectorize_batch、save、prepare_corpus、main
dataset/summary_tree.py	postorder、render_section、summary_dependencies、section_inputs
double_layer_plan.py	DAGNode、SkeletonGraph、Concretization、DoubleLayerPlan、parse_double_layer_plan、plan_steps_from_raw
legal_agent/__init__.py	包初始化、导出或命名空间
legal_agent/cache.py	SemanticCache、UnifiedQueryRouter_Soul
legal_agent/compat.py	说明、常量、兼容层或显式脚本；见对应功能章节
legal_agent/demo.py	main
legal_agent/evidence.py	source_audit
legal_agent/gateway.py	classify
legal_agent/llm.py	model_role、create_client、complete
legal_agent/nodes.py	node_l0_gateway、node_planner、node_executor、node_replanner、node_generate、node_write_code、node_execute_code、node_inject_calc_result、node_cleanup_sandbox、node_await_input
legal_agent/operators.py	LegalLLMBase、ExecutionPlan、AgenticNodesOperator
legal_agent/reasoning.py	Reasoner
legal_agent/retrieval.py	说明、常量、兼容层或显式脚本；见对应功能章节
legal_agent/routing.py	UnifiedQueryRouter_Query
legal_agent/scheduling.py	tasks、validate_plan、user_facts_for、execute_ready、replan_pending
legal_agent/state.py	AgentState
legal_agent/workflow.py	build_plan_replan_agent
model/Unsloth.py	main
model/__init__.py	包初始化、导出或命名空间
model/behavior.py	prepare、run、main
model/data/__init__.py	包初始化、导出或命名空间
model/data/evol_instruct.py	synthesize_batch、generate_noisy_queries、generate_replanner_scenarios、main
model/data/synthetic/__init__.py	包初始化、导出或命名空间
model/data/synthetic/__main__.py	main

文件	主要名字 / 用途
model/data/synthetic/candidates.py	CandidateProvider、pending_candidate
model/data/synthetic/contracts.py	Contract、Config、Evidence、Record、Outcome、Task、Step、Trajectory、Source、SeedGroup、Draft
model/data/synthetic/export.py	dumps、validate_seeds、export_dataset
model/data/synthetic/generation.py	split_for、weighted_count、generate_groups、trajectory_candidates
model/data/synthetic/simulation.py	resolve、oracle、Environment、reference、score
model/tool_schema.py	main
model/training/__init__.py	包初始化、导出或命名空间
model/training/train_extractor_grader.py	顶层训练脚本，见第 18 节
model/training/train_meta_planner.py	SemanticDTWRewarder
model/training/train_reasoner.py	顶层训练脚本，见第 18 节
model/training/train_replanner_grpo.py	GRPOTournamentEvaluator、reward_func
model/training/train_retriever.py	顶层训练脚本，见第 18 节
model/utils/__init__.py	包初始化、导出或命名空间
model/utils/unsloth_loader.py	UnslothLoader、load_planner、load_replanner、load_extractor_grader、load_reasoner
model/utils/vllm_engine.py	VLLMPromptCacheEngine、create_llm_backend
multiple-search/SemanticCache/engine.py	InMemoryVectorStore、RedisVectorStore、build_semantic_cache
multiple-search/legal_sandbox/example_usage.py	get_manager、node_write_code、node_execute_code、terminate_session
multiple-search/legal_sandbox/sandbox_exec.py	兼容导出 / 显式示例入口
multiple-search/legal_sandbox/sandbox_manager.py	兼容导出 / 显式示例入口
multiple-search/legal_sandbox/sandbox_server.py	兼容导出 / 显式示例入口
multiple-search/soul.py	兼容导出 / 显式示例入口
observability.py	JsonFormatter、setup_structured_logging、MetricsCollector、get_metrics、HealthHandler、start_health_server、retry_with_backoff、ResilientLLMClient、DeadLetterQueue、get_dlq、push_to_dlq、LLMTimer
prompts.py	build_meta_planner_messages、build_extractor_messages、build_grader_messages、build_reasoner_messages、format_evidence_chain、build_generator_messages、build_code_generator_messages、build_replanner_messages、build_router_l2_messages、build_benchmark_judge_messages、build_causal_extraction_messages、build_causal_verification_messages、get_prompt、list_prompts

文件	主要名字 / 用途
query.py	兼容导出 / 显式示例入口
replanner_rules.py	task_has_kw、apply_hard_rules、list_rules
replanner_rules_report.py	说明、常量、兼容层或显式脚本；见对应功能章节
spikes/spike_checkpoint_resume.py	state_path、TaskLedger、Cycle、node_plan、node_dispatch、node_finish、build_graph、make_saver、Bus、phase_suspend、phase_resume、Check、main
tools/audit_entrypoints.py	EntryPoint、probe_import、error_excerpt、main
tools/calibrate_causal_support.py	main
tools/ci.py	main
tools/long_term_memory.py	main
training_data_guide.py	说明、常量、兼容层或显式脚本；见对应功能章节

下面是其他运行、部署和打包文件：

文件	用途
.continue/rules/new-rule.md	编辑器规则占位文件，不是业务实现
.env.example	环境变量声明，不含实际凭据
.gitattributes	文本与 Git 属性
.gitignore	运行和敏感产物排除
MANIFEST.in	发行包清单
asynchronization/Dockerfile.worker	Worker 镜像构建声明
asynchronization/entrypoint.sh	新运行时 / 旧阶段 Worker 显式选择
asynchronization/explain	旧异步链说明
asynchronization/k8s_worker_deployment.yaml	旧阶段链 Kubernetes 示例
benchmark_causal/data/legal_corpus.jsonl	版本化法条评测语料
config.yaml	部署配置真源
dataset/material_rules.json	声明式加工深度规则
multiple-search/legal_sandbox/Dockerfile	代码沙箱镜像构建声明
pyproject.toml	包、入口和依赖声明
pytest.ini	测试发现与 pytest 选项
requirements-behavior.txt	可选固定版本行为优化依赖

文件	用途
requirements-dev.txt	开发和测试依赖
requirements-ingestion.txt	资料规则及可选 PDF 检查依赖
requirements-runtime.constraints.txt	已有直接运行依赖约束
requirements-train.txt	训练及本地推理依赖
requirements.txt	运行依赖
tools/run.ps1	D 盘运行与统一命令包装

26. 测试文件和已有检查记录

测试目录的 33 个 Python 文件都列在下面。这里数的是代码里的测试函数，一份函数可能对应多个参数化用例，所以不是 pytest 最终显示的用例数。这次没有重跑它们。

测试文件	测试函数定义数
tests/conftest.py	0
tests/harness.py	0
tests/test_agent_runtime.py	8
tests/test_agent_tools.py	8
tests/test_causal_corpus.py	15
tests/test_causal_evidence.py	29
tests/test_causal_gates.py	24
tests/test_causal_generator.py	17
tests/test_causal_pipeline.py	15
tests/test_causal_scm.py	22
tests/test_chain_integration.py	12
tests/test_corpus_ingest.py	25
tests/test_double_layer_plan.py	25
tests/test_e2e_graph.py	19
tests/test_graph_vertex_names.py	18
tests/test_long_term_memory.py	29
tests/test_material_routing.py	14
tests/test_memory_graph_bridge.py	31

测试文件	测试函数定义数
tests/test_memory_thresholds.py	21
tests/test_model_behavior.py	11
tests/test_module_boundaries.py	2
tests/test_pageindex.py	10
tests/test_pageindex_navigation.py	16
tests/test_pageindex_structure.py	12
tests/test_parallel_agent.py	23
tests/test_replanner_rules.py	18
tests/test_runtime_api_transport.py	4
tests/test_runtime_recovery.py	4
tests/test_runtime_store.py	12
tests/test_sandbox.py	36
tests/test_schema_alignment.py	12
tests/test_space_sandbox.py	42
tests/test_synthetic_component.py	13

各功能的说明和以前的检查记录

下面这些文件都已存在。看测试结果时，要一起看日期、当时的代码版本，以及哪些地方用了替身；以前测过，不等于新代码或真实服务也测过。

- docs/AGENT_RUNTIME.md
- docs/CAUSAL_EVIDENCE.md
- docs/CONTRACTS.md
- docs/LONG_TERM_MEMORY.md
- docs/MATERIAL_ROUTING.md
- docs/MODEL_BEHAVIOR.md
- docs/MODULAR_ARCHITECTURE.md
- docs/PAGEINDEX_IMPLEMENTATION.md
- docs/PAGEINDEX_INTEGRATION_REVIEW.md
- docs/PARALLEL_AGENT.md
- docs/PERFORMANCE_PATHS.md
- docs/PROJECT_OVERVIEW.md

- docs/RESUME_REMINDERS.md
- docs/SANDBOX_BACKENDS.md
- docs/SYNTHETIC_DATA.md
- docs/TOOL_SCHEMA_ALIGNMENT.md
- docs/archive/README_before_modularization.md
- docs/validation/agent_runtime_2026-10-18.txt
- docs/validation/causal_evidence_ci_2026-10-18.txt
- docs/validation/chain_active_ci_2026-10-18.txt
- docs/validation/long_term_memory_ci_2026-10-18.txt
- docs/validation/material_routing_2026-10-20.txt
- docs/validation/model_behavior_2026-11-04.txt
- docs/validation/pageindex_a_2026-10-18.txt
- docs/validation/pageindex_b_2026-10-18.txt
- docs/validation/pageindex_c_2026-10-18.txt
- docs/validation/parallel_agent_2026-10-18.txt
- docs/validation/sandbox_space_2026-10-19.txt
- docs/validation/synthetic_component_2026-10-21.txt
- docs/validation/tool_schema_alignment_2026-11-07.txt

这份清单对照了 Git 文件列表、Python AST、源码、配置和已有说明。没有读取或公开凭据、运行数据库正文、模型权重，也没动你未跟踪的工作区文件。清单对应开头写明的代码版本，以后功能有变化时，也要跟着更新。